

ACTIVIDAD II – RESPONSABILIDAD EN PROYECTOS TECNOLÓGICOS

1.1. Responsabilidad en el desarrollo de software

Cuando desarrollamos software, es muy común pensar que nuestro trabajo termina en el momento en que el sistema funciona correctamente.

Que si compila, si corre, si cumple con lo que pedía el cliente... ya está.

Pero en la práctica, la realidad es otra.

Un sistema no vive solo en nuestro entorno de desarrollo.
Se usa, se modifica, interactúa con personas y con otros sistemas.

Y en ese contexto, pueden aparecer problemas:

- errores que no se detectaron a tiempo
- fallas en producción
- o situaciones que no fueron previstas durante el desarrollo

Y es en ese momento —cuando el sistema deja de ser solo código y empieza a generar impacto real—

donde aparece una pregunta clave:

¿Quién responde?

1.2. ¿Cuándo aparece la responsabilidad?

La responsabilidad no aparece simplemente porque exista un sistema o porque haya código.

Aparece cuando ese sistema tiene un impacto real.

Mientras el software está en desarrollo, los errores forman parte del proceso.

Pero cuando ese sistema se utiliza en un entorno real —con usuarios, datos y decisiones—

las consecuencias empiezan a ser diferentes.

¿En qué situaciones aparece?

La responsabilidad comienza a ser relevante cuando ocurren situaciones como:

- un sistema genera un daño
- se producen pérdidas económicas
- un usuario o cliente se ve afectado

Ejemplo

Imaginemos un sistema de gestión que utiliza una empresa para almacenar información importante.

El sistema falla...
y se pierden datos críticos.

En ese momento, el problema deja de ser técnico.

Ya no se trata solo de un error en el código.

Se trata de una situación que tiene consecuencias reales.

Y es ahí donde vuelve a aparecer la pregunta clave:

¿Quién responde?

1.3. Tipos de responsabilidad

No todas las situaciones problemáticas en un proyecto tecnológico son iguales.

Dependiendo del contexto, pueden aparecer distintos tipos de responsabilidad.

Entender esto es importante porque no es lo mismo un error técnico...
que un incumplimiento de contrato o una mala práctica profesional.

Responsabilidad civil

Este tipo de responsabilidad aparece cuando se genera un **daño económico**.

Es decir, cuando un sistema o desarrollo provoca pérdidas a otra persona o empresa.

Ejemplo

Una empresa contrata a un desarrollador para crear un sistema de facturación.

El sistema tiene errores y genera cálculos incorrectos.

Como consecuencia, la empresa pierde dinero.

En este caso:

- hay un daño económico
- alguien resultó perjudicado
- y ese daño puede ser reclamado

Idea clave

Si un sistema genera pérdidas... puede haber responsabilidad civil.

Responsabilidad contractual

Este tipo de responsabilidad aparece cuando existe un **acuerdo previo** entre las partes.

Es decir, cuando hay algo que se prometió... y no se cumple.

Ejemplo

Un desarrollador acuerda entregar un sistema en una fecha determinada.

No cumple con el plazo...

o entrega un producto distinto al pactado.

En este caso:

- existe un acuerdo
- hay un incumplimiento
- y puede haber consecuencias legales

Idea clave

Si no cumplís lo que acordaste... hay responsabilidad contractual.

Responsabilidad profesional

Este tipo de responsabilidad está relacionada con **cómo se ejerce la profesión**.

No se trata solo de cumplir o no...
sino de hacerlo de manera adecuada.

Ejemplo

Un desarrollador implementa un sistema sin realizar pruebas suficientes.

El sistema falla en producción...
y afecta a los usuarios.

En este caso:

- no hubo intención de dañar
- pero sí hubo negligencia
- es decir, falta de cuidado o profesionalismo

Idea clave

No solo importa hacer el trabajo...
sino hacerlo bien.

En los proyectos tecnológicos, los problemas no siempre son técnicos.

Muchas veces, se convierten en problemas legales dependiendo del contexto.

Y entender qué tipo de responsabilidad está en juego
es parte del crecimiento profesional como desarrollador.

1.4. Factores que influyen en la responsabilidad

Cuando ocurre un problema en un sistema, lo primero que suele pensarse es:

“Hubo un error.”

Pero en el análisis real, eso no alcanza.

La responsabilidad no se define únicamente por el error en sí, sino por todo el contexto en el que ese error ocurrió.

¿Qué significa esto?

Que ante una misma falla, la responsabilidad puede cambiar dependiendo de distintos factores.

No es lo mismo un error en un proyecto personal que en un sistema utilizado por una empresa con clientes reales.

Factores que influyen

1. Si existe un contrato

Un contrato define:

- qué se debía hacer
- en qué condiciones
- con qué alcance

Ejemplo

Un desarrollador acuerda entregar un sistema con ciertas funcionalidades.

El sistema no cumple con lo pactado.

En este caso, el contrato es clave para determinar la responsabilidad.

2. Cómo está organizado el proyecto

La estructura del proyecto influye directamente en quién responde.

Ejemplo

Un sistema falla en una startup organizada como sociedad.

Puede responder la empresa.

El mismo sistema desarrollado por una persona freelance:

Puede responder directamente el desarrollador.

3. Si hubo negligencia

La negligencia implica falta de cuidado o de buenas prácticas.

Ejemplo

Un desarrollador implementa un sistema sin realizar pruebas suficientes.

El sistema falla en producción.

No fue intencional, pero sí evitable.

4. Si se tomaron decisiones correctas

Muchas veces, los problemas no surgen del código...
sino de decisiones mal tomadas.

Ejemplo

Un equipo decide no documentar el sistema para avanzar más rápido.

Luego, ante un error, nadie sabe cómo solucionarlo.

En este caso, el problema no fue técnico...
sino organizativo.

La responsabilidad no depende solo de lo que pasó...
sino de cómo se trabajó.

Por eso, como desarrolladores:

- no solo escriben código
- también toman decisiones

Y esas decisiones pueden reducir o aumentar los riesgos.

Después de ver que los proyectos pueden generar problemas y responsabilidades,
aparece una pregunta lógica:

¿Se pueden evitar estos problemas?

La respuesta es: **no se pueden eliminar por completo... pero sí se pueden reducir.**

Y eso depende, en gran parte, de cómo se trabaja.

Buenas prácticas en proyectos tecnológicos

Existen ciertas prácticas que ayudan a minimizar riesgos y a trabajar de manera más profesional.

Definir contratos claros

Un contrato no es solo un requisito formal.

Es una herramienta que permite dejar en claro:

- qué se va a desarrollar
- qué incluye el trabajo
- qué no está incluido
- cuáles son los plazos

Ejemplo

Un cliente pide una aplicación, pero no se define el alcance.

Luego solicita cambios constantes que no estaban previstos.

Con un contrato claro, esto se puede evitar.

Documentar el trabajo

Documentar significa dejar registro de lo que se hace.

No solo para otros... también para uno mismo.

Ejemplo

Un sistema falla y nadie recuerda cómo estaba implementado.

La falta de documentación aumenta el riesgo.

Establecer roles

En equipos de desarrollo, no todos hacen lo mismo.

Definir roles permite:

- organizar el trabajo
- evitar superposición de tareas
- reducir conflictos

Ejemplo

Dos desarrolladores modifican la misma parte del sistema sin coordinar.

El problema no es técnico... es organizativo.

Testear sistemas

Las pruebas son una parte fundamental del desarrollo.

Permiten detectar errores antes de que el sistema esté en uso real.

Ejemplo

Un sistema se lanza sin pruebas suficientes y falla en producción.

Ese error podría haberse evitado.

Trabajar bien no significa solo que el código funcione...
significa trabajar de manera ordenada y profesional.

Estas prácticas no eliminan todos los riesgos,
pero ayudan a:

- prevenir problemas
- reducir responsabilidades
- mejorar la calidad del proyecto

Y eso es parte del perfil de un desarrollador profesional.

Programar no solo implica crear software...
también implica asumir consecuencias.

CASO

Un desarrollador freelance crea un sistema para una empresa.

El sistema falla y genera pérdidas económicas importantes.

1. ¿Existe responsabilidad?

👉 **Sí, existe responsabilidad.**

🧠 **Fundamentación:**

El sistema generó un daño concreto:

- pérdidas económicas
- impacto directo en una empresa

👉 Esto implica que la situación ya no es solo técnica, sino también legal.

Cuando hay daño, aparece la responsabilidad.

2. ¿Qué tipo de responsabilidad podría aplicarse?

👉 **Principalmente responsabilidad civil y posiblemente contractual.**

⚖️ **Responsabilidad civil**

✓ Aplica porque:

- hubo un daño económico
- hay una parte perjudicada

👉 El desarrollador podría estar obligado a reparar ese daño.

⚖️ **Responsabilidad contractual (si existía contrato)**

✓ Aplica si:

- había un acuerdo previo
- no se cumplió lo pactado

👉 Ejemplo:

- el sistema debía funcionar bajo ciertas condiciones
- no cumplió con esos requisitos

⚠️ También puede haber responsabilidad profesional

Si el error se debió a:

- falta de pruebas
- malas prácticas
- negligencia

3. ¿Influye si había contrato?

👉 Sí, influye directamente.

🧠 Con contrato

✓ Permite:

- definir qué se debía hacer
- establecer responsabilidades
- limitar el alcance del trabajo
- incluso limitar la responsabilidad

👉 Ejemplo:

Un contrato puede incluir cláusulas como:

- “no se garantiza funcionamiento absoluto”
- “responsabilidad limitada al monto del contrato”

✗ Sin contrato

👉 Todo se vuelve más ambiguo:

- no está claro qué se prometió
- no hay límites definidos
- aumenta el conflicto

El contrato no evita el problema...
pero ayuda a resolverlo.

4. ¿Cómo podría haberse evitado el problema?

No siempre se puede evitar completamente, pero sí reducir el riesgo.

Medidas clave:

Definir un contrato claro

- alcance del sistema
- funcionalidades
- límites
- responsabilidades

Realizar pruebas (testing)

- pruebas antes de producción
- validación con el cliente

Documentar el sistema

- funcionamiento
- decisiones técnicas
- cambios realizados

Establecer buenas prácticas

- control de versiones
- revisiones de código
- entornos de prueba

Conclusión del caso

El problema no surge solo del error técnico...
sino de cómo se gestionó el proyecto.

En el desarrollo profesional:

- no alcanza con que el sistema funcione
- también importa cómo se construye y cómo se acuerda